25
고급 RAG 기법

RAG의 답은 찾아온 문서만큼만 좋습니다. 찾아온 후보를 다시 채점해 순서를 바꾸는 리랭킹 두 가지와, 질문 대신 가상의 답으로 검색하는 하이드(HyDE)를 만들어 봅니다.

오늘 다루는 것

1. LLM 기반의 리랭킹
2. 크로스 인코더 기반의 리랭킹
3. 검색 성능을 높이는 하이드(HyDE)

끝나고 나면 할 수 있는 것

1. "넓게 찾고 좁게 다시 고른다"는 2단계 검색의 구조와 이유를 설명할 수 있다.
2. 이중 인코더와 크로스 인코더의 차이를 입력의 모양과 속도로 비교할 수 있다.
3. 하이드가 질문 대신 무엇으로 검색하는지, 언제 도움이 되고 언제 위험한지 말할 수 있다.

2026년 10월 16일 금요일 14:30 (10/16 자료 5/5)
24장과 같이 OpenAI API 대신 내 컴퓨터에서 도는 모델을 썼습니다. 실습 2의 리랭커 모델(BAAI/bge-reranker-v2-m3)은 교재와 같은 것이고 약 2.3GB입니다.

25

고급 RAG 기법

RAG의 성능은 두 가지가 정합니다. 알맞은 문서를 찾아오는 검색기, 그리고 그것을 보고 답을 쓰는 LLM입니다. 이 장은 검색기를 고칩니다. LLM이 아무리 좋아도 엉뚱한 문서를 받으면 엉뚱한 답을 하거나 "모른다"고 답할 수밖에 없기 때문입니다.

왜 배우나: 오늘의 학습 경로

벡터 검색은 빠르지만 거칠다
24장의 검색

후보를 LLM이 다시 채점한다
LLM 리랭킹

전용 모델이 다시 채점한다
크로스 인코더

질문을 문서처럼 바꾼다
하이드

세 방법 모두 24장의 금융 챗봇에 쓴 한국은행 「경제금융용어 700선」으로 실습합니다. 리랭킹은 검색의 **뒤**를 고치고, 하이드는 검색의 **앞**(무엇으로 검색할지)을 고칩니다.

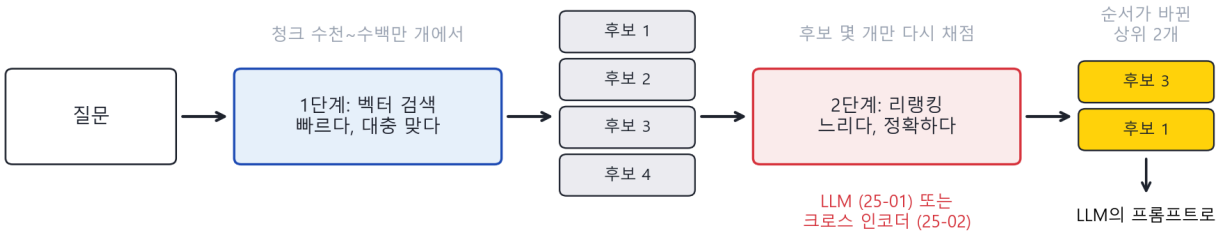


그림 1 2단계 검색. 빠른 방법으로 넓게 찾고, 정확한 방법으로 몇 개만 다시 고른다

준비

공통 준비물: 벡터 데이터베이스와 LLM

핵심 한 줄

세 실습이 같은 벡터 데이터베이스와 같은 LLM을 쓰므로, 한 파일(rag_base.py)에 만들어 두고 불러 쓴다.

정의

함수	하는 일	교재에서 대응하는 것
`load_vector_store()`	PDF를 300자 청크(100자 겹침)로 잘라 임베딩해 크로마에 넣는다. 한 번 만들면 폴더에 저장해 두고 다시 쓴다	Chroma.from_documents(chunks, OpenAIEmbeddings())
`ask(prompt)`	프롬프트를 로컬 LLM에 넣어 답을 받는다	llm.invoke(prompt).content (gpt-4o)
`generate_answer(query, documents)`	검색된 문서를 근거로 최종 답을 쓰게 한다	같은 이름의 함수

청크를 24-07(쪽 단위, 약 1,000자)보다 작게 300자로 자릅니다. 청크가 작으면 한 청크에 한 가지 이야기만 들어가 검색이 정밀해지고, 리랭킹으로 고를 후보도 많아집니다.

예시 적용

용어집 PDF에서 머리말, 목차, 맺음말을 뺀 쪽들을 300자(100자 겹침)로 자르니 청크가 **1,965개** 나왔습니다. 24-07에서는 같은 문서가 352개(쪽 단위)였습니다.

"디커플링이란 무엇인가?"로 검색하면 114쪽의 청크 둘이 나옵니다. 첫째는 디커플링 설명의 앞부분, 둘째는 뒷부분입니다. 한 용어의 설명이 청크 둘로 나뉘어 들어가 있습니다.

실습 0

rag_base.py: 세 실습이 함께 쓰는 벡터 데이터베이스와 LLM

```python
# 25장 실습의 공통 준비물: 벡터 데이터베이스와 로컬 LLM (API 키 불필요)
# 코랩에서는 먼저:  !pip install langchain-community langchain-chroma pypdf
import os
import urllib.request

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_core.embeddings import Embeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from transformers import AutoModelForCausalLM, AutoTokenizer

FILE = "2020_경제금융용어 700선_게시.pdf"
DB_DIR = "./finance_db"

class LocalEmbeddings(Embeddings):                    # 교재의 OpenAIEmbeddings 자리
    def __init__(self, name="jhgan/ko-sroberta-multitask"):
        self.model = SentenceTransformer(name)
        self.model.max_seq_length = 512

    def embed_documents(self, texts):
        return self.model.encode(texts, normalize_embeddings=True).tolist()

    def embed_query(self, text):
        return self.embed_documents([text])[0]

def load_vector_store():
    """PDF를 300자 청크(100자 겹침)로 잘라 크로마에 넣고, 다음부터는 불러 쓴다."""
    embeddings = LocalEmbeddings()
    if os.path.exists(DB_DIR):
        return Chroma(persist_directory=DB_DIR, embedding_function=embeddings,
                      collection_name="finance")
    urllib.request.urlretrieve(
        "https://github.com/chatgpt-kr/openai-api-tutorial/raw/main/ch07/"
        "2020_%EA%B2%BD%EC%A0%9C%EA%B8%88%EC%9C%B5%EC%9A%A9%EC%96%B4%20700"
        "%EC%84%A0_%EA%B2%8C%EC%8B%9C.pdf", filename=FILE)
    pages = PyPDFLoader(FILE).load()[17:-1]           # 머리말, 목차, 맺음말은 뺀다
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=300,
                                                   chunk_overlap=100)
```

```
    chunks = text_splitter.split_documents(pages)
    return Chroma.from_documents(chunks, embeddings, collection_name="finance",
                                 persist_directory=DB_DIR)

_llm = {}

def ask(prompt, max_new_tokens=120):                # 교재의 llm.invoke(prompt) 자리
    if not _llm:                                    # 처음 부를 때만 모델을 올린다
        name = "Qwen/Qwen2.5-1.5B-Instruct"
        _llm["tok"] = AutoTokenizer.from_pretrained(name)
        _llm["model"] = AutoModelForCausalLM.from_pretrained(name)
    enc = _llm["tok"].apply_chat_template(
        [{"role": "user", "content": prompt}], add_generation_prompt=True,
        return_tensors="pt", return_dict=True)
    out = _llm["model"].generate(**enc, max_new_tokens=max_new_tokens,
                                 do_sample=False)
    new_tokens = out[0, enc["input_ids"].shape[1]:]
    return _llm["tok"].decode(new_tokens, skip_special_tokens=True).strip()

def generate_answer(query, documents):
    """검색된 문서를 근거로 최종 답변을 만든다."""
    context = "\n\n".join(doc.page_content for doc in documents)
    return ask(f"다음 검색 결과를 바탕으로 질문에 두 문장 이내로 답하세요.\n\n"
               f"검색 결과:\n{context}\n\n질문: {query}\n\n답변:")

def show(documents, width=74):
    for i, doc in enumerate(documents, 1):
        text = doc.page_content.replace("\n", " ")
        print(f"  문서 {i} [{doc.metadata['page']}쪽] {text[:width]}")

if __name__ == "__main__":
    vector_store = load_vector_store()
    print("벡터 데이터베이스에 든 청크:", vector_store._collection.count())
    show(vector_store.similarity_search("디커플링이란 무엇인가?", k=2))
```

```
실행 결과
벡터 데이터베이스에 든 청크: 1965
  문서 1 [114쪽] 디커플링(decoupling)이란 탈동조화라고 번역할 수 있는데, 어떤 나라나 지역의 경제 가 인접한 다른
국가나 전반적인 세계 경제
  문서 2 [114쪽] 일부 경제 변수의 흐름이 국가 간 또는 특정국가내에서 서로 다른 흐름을 보이는 현상도  디커플링이라고
할 수 있다. 국가 간의 경우
```

교재에는 없는 파일입니다. 교재는 절마다 같은 준비 코드를 되풀이하는데, 여기서는 한 파일로 모았습니다. 데이터도 바꿨습니다. 교재의 25장은 「2023 북한인권보고서」를 쓰지만, 이 자료에서 쓰는 로컬 LLM(Qwen2.5)이 시험해 본 그 주제의 질문에 답을 거부해 실습을 이어 갈 수 없었습니다. 그래서 24-07과 같은 한국은행 「경제금융용어 700선」으로 바꿨습니다.

25-01 LLM 기반의 리랭킹

핵심 한 줄

리랭킹은 벡터 검색이 찾아온 후보를 더 정확한 방법으로 다시 채점해 순서를 바꾸는 것이고, LLM 기반 리랭킹은 그 채점을 LLM에게 프롬프트로 시킨다.

왜 배우나

벡터 검색은 질문과 문서를 따로 벡터로 바꿔 비교합니다. "비슷한 말이 나오는 문서"는 잘 찾지만 "질문에 답하는 문서"인지는 잘 가리지 못합니다. 가장 손쉬운 보완책이 이미 갖고 있는 LLM에게 "이 문서가 이 질문과 얼마나 관련 있나"를 물어보는 것입니다.

1) 리랭킹과 LLM 채점

정의

- 리랭킹(reranking, 재순위화): 1차 검색이 돌려준 후보 문서들을 다른 모델로 다시 평가해 순서를 다시 매기는 것. 보통 상위 몇 개만 남깁니다.
- LLM 기반 리랭킹: 후보 문서 하나하나에 대해 LLM에게 질문과의 관련성 점수(예: 1~10점)를 매기게 하고, 점수가 높은 순으로 다시 줄 세우는 방법.

후보가 k개이고 그중 m개를 남긴다면:

$$\text{점수}_i = \text{LLM}(\text{채점 프롬프트}(q,\ d_i)) \quad (i = 1, \ldots, k), \qquad \text{결과} = \text{점수가 높은 } m\text{개}$$

기호	뜻	이 실습에서
q	사용자의 질문	
d_i	벡터 검색이 찾아온 i번째 후보 문서	$k = 4$
m	리랭킹 뒤에 남기는 문서의 수	$m = 2$

	기본 RAG	LLM 리랭킹을 넣은 RAG
벡터 검색	문서 4개	문서 4개
채점	없음	LLM 호출 4회 (문서마다 1회)
답 생성	LLM 호출 1회 (문서 4개로)	LLM 호출 1회 (상위 2개로)
LLM 호출 합계	1회	5회

정확해지는 대신 LLM 호출이 후보 수만큼 늘어납니다. 비용과 시간이 그만큼 듭니다.

채점 프롬프트. 기준(직접 답을 담고 있으면 8~10점, 관련된 맥락이면 4~7점, 무관하면 1~3점)과 주의사항("비슷한 단어가 나온다는 것만으로 높은 점수를 주지 말 것")을 적고, 답을 `{"relevance_score": 숫자}` 형식으로만 내라고 지시합니다. 프로그램이 답에서 숫자를 꺼내 써야 하기 때문입니다.

예시 적용

질문 "물가가 계속 떨어지는 현상을 뭐라고 해?"에 대해 벡터 검색이 찾은 4개를 로컬 LLM이 채점했습니다(실습 1).

벡터 검색 순위	청크	LLM의 점수	모델의 응답 그대로
1	[114쪽] 디플레이션 물가가 지속적으로 하락하는 현상을 말한다…	8.0	```` ```json {"relevance_score": 8} ``` ````
2	[229쪽] 경기가 침체하고 물가가 하락한다. … 디플레이션이 발생하면…	5.0	```` ```json {"relevance_score": 5} ``` ````

3	[114쪽] 화폐가치는 상승하고 반대로 물가는 하락하는 디플레이션이 발생한다는…	1.0	`{"relevance_score": 1}`
4	[229쪽] 중앙은행은 정책금리를 낮추고 유동성을 공급한다…	1.0	```` ```json {"relevance_score": 1}``` ````

- 정의가 들어 있는 1번에 가장 높은 점수를 줘 1위를 지켰고, 최종 답 "물가가 지속적으로 하락하는 현상을 '디플레이션'이라고 합니다."도 맞았습니다.
- 그러나 채점이 고르지는 않습니다. 3번도 디플레이션의 원인을 설명하는 같은 항목의 청크인데 1점을 받았고, 디플레이션의 영향을 말하는 2번이 5점을 받았습니다. 작은 LLM의 점수는 흔들립니다.
- **형식을 어겼습니다.** "백틱은 쓰지 마십시오"라고 했는데도 넷 중 셋에 백틱(`` ```json ``)을 붙였습니다. 교재처럼 `json.loads` 로 읽었다면 셋 다 오류가 나 기본값 5점이 됐을 것입니다. 정규식으로 숫자만 찾도록 바꾼 덕에 점수를 읽을 수 있었습니다. LLM의 출력을 프로그램이 받아 쓸 때는 형식이 어긋날 것을 늘 대비해야 합니다.
- **시간:** 채점 4회에 290초가 걸렸습니다(CPU, 다른 작업과 함께). 호출이 후보 수만큼 늘어나는 비용이 실제로 보입니다.

비유

서류 심사와 면접

지원자 수천 명을 모두 면접할 수는 없습니다. 서류로 빠르게 몇 명을 추린 다음, 그 몇 명만 한 사람씩 면접해 순위를 다시 매깁니다. 면접은 정확하지만 한 명마다 시간이 듭니다.

서류 심사 = = 1단계 벡터 검색 (빠르다, 거칠다)
면접 = = 2단계 리랭킹 (느리다, 정확하다)
면접관 = = LLM 또는 크로스 인코더
면접 인원수 = = 후보 문서의 수 k

실습 1

벡터 검색 4개를 LLM이 다시 채점해 2개 고르기

```
# LLM 기반 리랭킹: 벡터 검색이 찾은 4개를 LLM이 다시 채점해 2개만 남긴다
import re
import time

from rag_base import ask, generate_answer, load_vector_store, show

vector_store = load_vector_store()

class DocumentScorer:
    """LLM에게 문서와 질문의 관련성을 1~10점으로 묻는다."""

    def evaluate_document(self, query, content):
        prompt = f"""아래 주어진 질문과 문서의 관련성을 평가해주세요.

[평가 기준]
- 문서가 질문에서 요구하는 정보를 직접적으로 포함하면 8-10점
- 문서가 질문과 관련된 맥락을 포함하지만 직접적인 답이 아니면 4-7점
- 문서가 질문과 거의 관련이 없으면 1-3점

[주의사항]
- 단순히 비슷한 단어가 등장하는 것은 높은 점수의 근거가 될 수 없습니다
- 질문의 의도와 문맥을 정확히 파악하여 평가해주세요

질문: {query}
문서: {content}

응답은 반드시 다음 JSON 형식이어야 합니다. 백틱은 쓰지 마십시오.:
{{"relevance_score": float}}"""
        response = ask(prompt, max_new_tokens=20)
        # 작은 모델은 형식을 잘 어긴다. 형식이 흐트러져도 숫자만 찾는다
        found = re.search(r"\d+(\.\d+)?", response)
        return float(found.group()) if found else 5.0, response

    def postprocess_documents(self, documents, query):
        scored_docs = []
        for i, doc in enumerate(documents, 1):
            score, raw = self.evaluate_document(query, doc.page_content)
            print(f"  문서 {i} → {score}점   (모델의 응답: {raw!r})")
            scored_docs.append((doc, score))
        ranked_docs = sorted(scored_docs, key=lambda x: x[1], reverse=True)
        return [doc for doc, _ in ranked_docs[:2]]   # 점수가 높은 2개만
```

```python
class SemanticRanker:
    """벡터 검색(1차) 뒤에 LLM 채점(2차)을 붙인 2단계 검색기."""

    def __init__(self, vector_store, scorer):
        self.vector_store = vector_store
        self.scorer = scorer

    def retrieve(self, query):
        vector_results = self.vector_store.similarity_search(query, k=4)
        print("═══ 벡터 검색 결과 (Top 4) ═══")
        show(vector_results)
        print("═══ LLM의 채점 (LLM 호출 4회) ═══")
        reranked_results = self.scorer.postprocess_documents(vector_results, query)
        print("═══ LLM의 리랭킹 결과 (Top 2) ═══")
        show(reranked_results)
        return reranked_results

ranker = SemanticRanker(vector_store, DocumentScorer())
query = "물가가 계속 떨어지는 현상을 뭐라고 해?"
print("질문:", query)
start = time.time()
documents = ranker.retrieve(query)
print(f"(리랭킹까지 {time.time() - start:.0f}초)")
print("\n최종 답변:", generate_answer(query, documents))
```

실행 결과

```
질문: 물가가 계속 떨어지는 현상을 뭐라고 해?
═══ 벡터 검색 결과 (Top 4) ═══
  문서 1 [114쪽] 디플레이션 물가가 지속적으로 하락하는 현상을 말한다. 디플레이션(deflation) 하에서는 물가상 승률이 마이너스로 하락하는 인플레
  문서 2 [229쪽] 경기가 침체하고 물가가 하락한다. 만약 경기가 계속 침체하여 디플레이션이 발생하면  명목이자율이 일정할 경우 실질이자율이 상승한다.
  문서 3 [114쪽] 화폐가치는 상승하고 반대로 물가는 하락하는 디플레이션이 발생한다는 점에 이견이  없다. 디플레이션이 발생하면 통화의 가치는 상승하고
  문서 4 [229쪽] 중앙은행은 정책금리를 낮추고 유동성을 공급한다. 그러나 금리를 계속 낮추는데도  경기가 회복되지 않고 이 이후에 명목이자율을 더 이상
═══ LLM의 채점 (LLM 호출 4회) ═══
  문서 1 → 8.0점   (모델의 응답: '```json\n{\n  "relevance_score": 8\n}\n```')
  문서 2 → 5.0점   (모델의 응답: '```json\n{\n  "relevance_score": 5\n}\n```')
  문서 3 → 1.0점   (모델의 응답: '{"relevance_score": 1}')
  문서 4 → 1.0점   (모델의 응답: '```json\n{"relevance_score": 1}\n```')
═══ LLM의 리랭킹 결과 (Top 2) ═══
  문서 1 [114쪽] 디플레이션 물가가 지속적으로 하락하는 현상을 말한다. 디플레이션(deflation) 하에서는 물가상 승률이 마이너스로 하락하는 인플레
  문서 2 [229쪽] 경기가 침체하고 물가가 하락한다. 만약 경기가 계속 침체하여 디플레이션이 발생하면  명목이자율이 일정할 경우 실질이자율이 상승한다.
(리랭킹까지 290초)

최종 답변: 물가가 지속적으로 하락하는 현상을 '디플레이션'이라고 합니다.
```

직접 해 보기 `query` 를 바꿔 보세요. LLM이 준 점수가 내가 문서를 읽고 매긴 순서와 같은가요? 채점 프롬프트의 평가 기준을 지우면 점수가 어떻게 달라지나요?

25-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 리랭킹은 벡터 데이터베이스의 모든 문서를 LLM으로 다시 채점하는 것이다. (O / X) 이름 _____

2 LLM 리랭킹을 넣으면 LLM 호출 횟수가 늘어난다. (O / X) 이름 _____

3 후보 4개를 LLM으로 리랭킹한 뒤 답까지 만들 때 LLM 호출은 모두 몇 회인가? 이름 _____

① 1회 ② 4회 ③ 5회

4 채점 결과를 JSON 형식으로 내라고 지시하는 이유는? 이름 _____

① 답이 더 정확해져서 ② 프로그램이 점수를 꺼내 쓰기 쉬워서 ③ 토큰이 줄어서

5 1차 검색의 후보를 다시 평가해 순서를 바꾸는 것을 _____이라고 한다. 이름 _____

6 실습에서 리랭킹 뒤에 남긴 문서는 _____개다. 이름 _____

7 실습의 작은 LLM이 지시한 출력 형식을 지키지 않았을 때 코드가 어떻게 대처했는지, 왜 그런 대비가 필요한지 설명해 보세요. 이름 _____

원문: 25-01 LLM 기반의 리랭킹 — https://wikidocs.net/287284 (교재는 gpt-4o로 채점하고 json.loads로 점수를 읽습니다. 여기서는 로컬 LLM을 썼고, 작은 모델이 형식을 어기는 경우에 대비해 정규식으로 숫자만 찾도록 바꿨습니다. 데이터와 질문은 위에서 밝힌 이유로 금융 용어집으로 바꿨습니다)

25-02 크로스 인코더 기반의 리랭킹

핵심 한 줄

이중 인코더는 질문과 문서를 따로 읽어 벡터끼리 비교하고, 크로스 인코더는 둘을 한 입력으로 함께 읽어 관련성 점수 하나를 낸다. 앞의 것으로 빠르게 추리고 뒤의 것으로 정확히 다시 고른다.

왜 배우나

LLM에게 채점을 시키면 느리고 비쌉니다. 채점만 하도록 학습된 훨씬 작은 모델이 크로스 인코더입니다. 20-02에서 두 문장을 [SEP]로 이어 넣었던 바로 그 입력 방식이고, 19-05에서 SBERT가 나온 이유(BERT로 문장 쌍을 하나씩 비교하면 너무 느리다)를 반대쪽에서 다시 보게 됩니다.

1) 이중 인코더와 크로스 인코더

정의

- 이중 인코더(bi-encoder): 질문과 문서를 **각각** 인코더에 넣어 벡터로 바꾼 뒤 코사인 유사도를 구하는 방식. 19-05의 SBERT, 24장의 벡터 검색이 모두 이것입니다.
- 크로스 인코더(cross-encoder): 질문과 문서를 `[CLS] 질문 [SEP] 문서 [SEP]` 로 **함께** 넣어, 관련성 점수 하나를 바로 내는 방식. 20-02의 문장 쌍 분류와 구조가 같습니다.

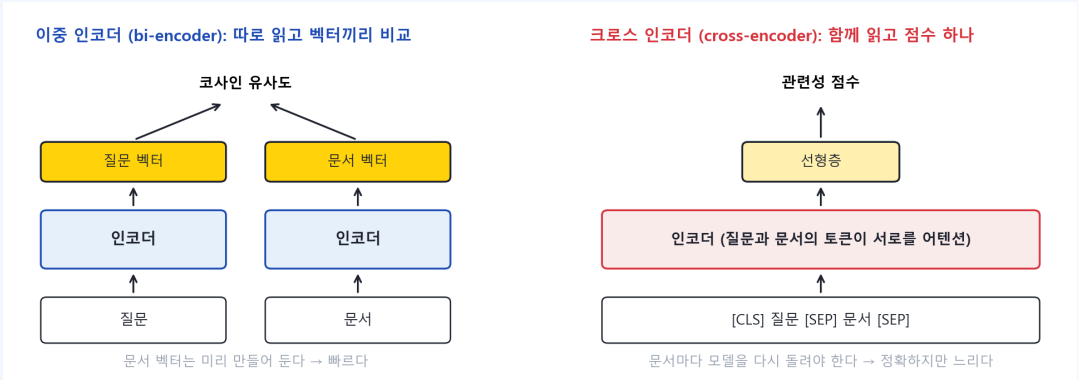


그림 2 이중 인코더는 문서 벡터를 미리 만들어 둘 수 있다. 크로스 인코더는 질문이 와야 계산을 시작할 수 있다

이중 인코더: $\text{점수}(q, d) = \cos\big(f(q),\ f(d)\big)$ 크로스 인코더: $\text{점수}(q, d) = g\big([q;\ d]\big)$

기호	뜻
f	글 하나를 벡터 하나로 바꾸는 인코더
g	질문과 문서를 이은 입력 $[q;\ d]$를 받아 점수 하나를 내는 모델

	이중 인코더	크로스 인코더
질문과 문서가 서로를 보는가	아니다. 각자 벡터가 된 뒤에야 만난다	그렇다. 셀프 어텐션에서 토큰끼리 직접 본다
문서 쪽 계산을 미리 해 둘 수 있는가	있다 (벡터 데이터베이스)	없다
문서 N개에서 검색할 때 모델을 돌리는 횟수	1번 (질문만)	N번
정확도	낮다	높다

그래서 크로스 인코더는 1단계 검색기로는 쓸 수 없고, 이중 인코더가 추려 준 몇 개~몇십 개만 다시 채점하는 2단계에 씁니다. 실습의 `BAAI/bge-reranker-v2-m3` 는 여러 언어를 지원하는 크로스 인코더 리랭커입니다.

예시 적용

작은 예: "고양이의 수명"(실습 2의 앞부분). 후보 문장 넷을 크로스 인코더에 질문과 쌍으로 넣었습니다.

후보 문장	관련성 점수
고양이는 보통 12년에서 18년 정도 산다.	0.999
강아지의 평균 수명은 10년에서 13년이다.	0.574
고양이는 하루에 열두 시간 넘게 잔다.	0.095
오늘 점심은 김치찌개를 먹었다.	0.000

"고양이"라는 단어가 든 문장(잠)보다 "수명"을 말하는 문장(강아지)에 더 높은 점수를 줬습니다. 질문이 묻는 것이 무엇인지를 보고 있다는 뜻입니다. 그리고 질문에 답하는 문장과 나머지의 점수 차이가 큽니다.

용어집에서: "물가가 계속 떨어지는 현상을 뭐라고 해?" 벡터 검색으로 20개를 찾고 크로스 인코더로 다시 줄 세웠습니다.

리랭킹 순위	점수	벡터 검색에서의 순위	청크의 내용
1	0.997	1위	"디플레이션 물가가 지속적으로 하락하는 현상을 말한다…"
2	0.969	8위	디커플링 설명의 끝부분 + "디플레이션 물가가 지속적으로 하락하는 현상을 말한다…"
3	0.742	5위	디스인플레이션(물가 상승률이 낮아지는 현상) 설명

- 1위는 그대로이고, 벡터 검색에서 8위였던 청크가 2위로 올라왔습니다. 출력에 보이는 앞부분은 디커플링 이야기라 엉뚱해 보이지만, 청크 전체를 출력해 보면 뒤쪽에 디플레이션의 정의가 들어 있습니다. 300자씩 자르다 보니 두 용어의 설명이 한 청크에 걸친 것입니다.
- 이중 인코더는 청크 전체를 벡터 하나로 뭉치므로 앞부분의 디커플링 이야기에 끌려가 8위를 준 것으로 보이고, 크로스 인코더는 질문과 함께 읽으며 답이 있는 부분을 알아봤습니다. 벡터 검색에서 2, 3위였던 청크(디플레이션이라는 말은 나오지만 정의는 아닌 부분)는 3위 밖으로 밀려났습니다.

잘 안 된 예: "은행이 망하면 내 예금은 얼마까지 돌려받을 수 있어?"

- 리랭킹 1위는 벡터 검색 1위와 같은 "뱅크런" 청크입니다. 이 청크에 "예금자보호법에 의해 5,000만 원까지의 예금을 보호"라는 답이 들어 있습니다. 순서는 맞았습니다.
- 그런데 점수가 0.080입니다. 답이 든 청크인데도 낮습니다. 청크의 대부분이 뱅크런 설명이고, 앞에 "122 경제금융용어 700선" 같은 쪽 머리말이 섞여 있습니다. 리랭커의 점수는 **순서를 정하는 데** 쓰는 값이고, 절대값을 "몇 점 이상이면 관련 있다"는 기준으로 쓰려면 내 데이터에서 따로 확인해야 한다는 것을 보여 줍니다.

시간. 벡터 검색으로 1,965개 중 20개를 찾는 데 0.3~0.8초, 그 20개를 크로스 인코더로 채점하는 데 25~28초가 걸렸습니다(다른 작업이 함께 돌던 CPU 기준. GPU에서는 훨씬 빠릅니다). 20개에 이만큼이니 1,965개 전부를 채점하는 것은 생각하기 어렵습니다. 2단계로 나누는 이유입니다.

실습 2

벡터 검색 20개를 크로스 인코더로 다시 줄 세우기

```
# 크로스 인코더 리랭킹: 질문과 문서를 한 입력으로 넣어 관련성 점수를 받는다
import time

from sentence_transformers import CrossEncoder

from rag_base import load_vector_store, show

vector_store = load_vector_store()

# 1) 이중 인코더와 크로스 인코더의 차이를 작은 예로 확인
cross_encoder = CrossEncoder("BAAI/bge-reranker-v2-m3")   # 교재와 같은 모델
query = "고양이의 수명"
candidates = ["고양이는 보통 12년에서 18년 정도 산다.",
              "고양이는 하루에 열두 시간 넘게 잔다.",
              "강아지의 평균 수명은 10년에서 13년이다.",
              "오늘 점심은 김치찌개를 먹었다."]
scores = cross_encoder.predict([(query, c) for c in candidates])
```

```python
print("질문:", query)
for candidate, score in sorted(zip(candidates, scores), key=lambda x: -x[1]):
    print(f"  {score:.3f}  {candidate}")

# 2) 2단계 검색: 벡터 검색으로 넓게 20개 → 크로스 인코더로 다시 줄 세워 3개
def rerank(query, k_first=20, k_final=3):
    start = time.time()
    first = vector_store.similarity_search(query, k=k_first)
    t_first = time.time() - start
    start = time.time()
    scores = cross_encoder.predict([(query, d.page_content) for d in first])
    t_rerank = time.time() - start
    order = sorted(range(len(first)), key=lambda i: -scores[i])
    return first, order, scores, t_first, t_rerank

for query in ["물가가 계속 떨어지는 현상을 뭐라고 해?",
              "은행이 망하면 내 예금은 얼마까지 돌려받을 수 있어?"]:
    first, order, scores, t_first, t_rerank = rerank(query)
    print(f"\n질문: {query}")
    print(f"═══ 벡터 검색 Top 3 (20개 찾는 데 {t_first:.2f}초) ═══")
    show(first[:3], width=60)
    print(f"═══ 크로스 인코더 리랭킹 Top 3 (20개 채점에 {t_rerank:.1f}초) ═══")
    for i in order[:3]:
        text = first[i].page_content.replace("\n", " ")
        print(f"  {scores[i]:.3f} (벡터 검색 {i + 1}위) {text[:52]}")
```

```
실행 결과
질문: 고양이의 수명
  0.999  고양이는 보통 12년에서 18년 정도 산다.
  0.574  강아지의 평균 수명은 10년에서 13년이다.
  0.095  고양이는 하루에 열두 시간 넘게 잔다.
  0.000  오늘 점심은 김치찌개를 먹었다.

질문: 물가가 계속 떨어지는 현상을 뭐라고 해?
═══ 벡터 검색 Top 3 (20개 찾는 데 0.77초) ═══
  문서 1 [114쪽] 디플레이션 물가가 지속적으로 하락하는 현상을 말한다. 디플레이션(deflation) 하에서는 물가상 승률이
  문서 2 [229쪽] 경기가 침체하고 물가가 하락한다. 만약 경기가 계속 침체하여 디플레이션이 발생하면  명목이자율이
일정할 경우
  문서 3 [114쪽] 화폐가치는 상승하고 반대로 물가는 하락하는 디플레이션이 발생한다는 점에 이견이  없다. 디플레이션이
발생하면
═══ 크로스 인코더 리랭킹 Top 3 (20개 채점에 25.1초) ═══
  0.997 (벡터 검색 1위) 디플레이션 물가가 지속적으로 하락하는 현상을 말한다. 디플레이션(deflation) 하에서는
  0.969 (벡터 검색 8위) 일부 경제 변수의 흐름이 국가 간 또는 특정국가내에서 서로 다른 흐름을 보이는 현상도  디커플
  0.742 (벡터 검색 5위) 2.0% 등으로 낮아지는 경우 매년 물가상승률이 0보다 크기 때문에 상품과 서비스의  가격 수

질문: 은행이 망하면 내 예금은 얼마까지 돌려받을 수 있어?
═══ 벡터 검색 Top 3 (20개 찾는 데 0.27초) ═══
  문서 1 [138쪽] 122 경제금융용어 700선 뱅크런 은행의 대규모 예금인출사태를 가리키는 말이다. 금융시장 상황이
불안하거나
  문서 2 [210쪽] 은행 및 상호저축은행중앙회이며, 예금, 적금, 부금, 수입보험료 및 확정기여형퇴직연금  적립금 등을
적용대상
  문서 3 [81쪽] 맡겨 잘 운용하여 경제적 이득을 얻는데 그 의의가 있다. 금전신탁은 수익자가 신탁재산 의 운용을
지정하느냐
═══ 크로스 인코더 리랭킹 Top 3 (20개 채점에 28.0초) ═══
  0.080 (벡터 검색 1위) 122 경제금융용어 700선 뱅크런 은행의 대규모 예금인출사태를 가리키는 말이다. 금융시장 상
  0.015 (벡터 검색 8위) 청산 대상 금융기관의 자산, 부채를 임시로 넘겨받아 예금, 출금 등의 업무를 대행하고  합병,
  0.011 (벡터 검색 4위) 194 경제금융용어 700선 영향력계수 ∅후방연쇄효과∅ 참조 예금보험제도 예금보험은 금융기관의
```

직접 해 보기 `k_first`를 4, 20, 50으로 바꿔 보세요. 리랭킹 1등이 벡터 검색에서 몇 위였는지, 채점 시간이 어떻게 늘어나는지 적어 보세요.

확인 문제

25-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 크로스 인코더는 문서의 벡터를 미리 계산해 저장해 둘 수 있다. (O / X) 이름 _____

2 24장에서 크로마로 한 벡터 검색은 이중 인코더 방식이다. (O / X) 이름 _____

3 크로스 인코더의 입력은? 이름 _____

① 질문만 ② 문서만 ③ 질문과 문서를 이은 것

4 문서 1,000개에서 크로스 인코더만으로 검색하려면 질문 하나에 모델을 몇 번 돌려야 하나? 이름 _____

① 1번 ② 2번 ③ 1,000번

5 질문과 문서를 각각 벡터로 바꿔 비교하는 방식을 _____ 인코더라고 한다. 이름 _____

6 실습에서 1단계 벡터 검색으로 추린 후보는 _____개다. 이름 _____

7 크로스 인코더가 이중 인코더보다 정확한 이유를 "질문과 문서가 언제 만나는가"로 설명해 보세요. 이름 _____

원문: 25-02 크로스 인코더 기반의 리랭킹 — https://wikidocs.net/287331 (교재는 랭체인의 HuggingFaceCrossEncoder, CrossEncoderReranker, ContextualCompressionRetriever로 4개를 2개로 줄이고 gpt-4o로 답을 만듭니다. 여기서는 같은 리랭커 모델을 sentence-transformers의 CrossEncoder로 직접 불러 점수를 눈으로 볼 수 있게 했고, 후보를 20개로 늘려 순서가 바뀌는 것을 봤습니다. "고양이의 수명" 예는 교재의 설명을 실제로 돌려 본 것입니다. 답 생성은 생략했습니다)

25-03 검색 성능을 높이는 하이드(HyDE)

핵심 한 줄

하이드는 질문을 바로 임베딩해 검색하지 않고, LLM에게 그 질문의 답처럼 보이는 가상의 문서를 먼저 쓰게 한 뒤 그 가상 문서를 임베딩해 검색한다.

왜 배우나

질문과 문서는 생김새가 다릅니다. 질문은 짧고 물음꼴이며 일상어로 쓰입니다. 문서는 길고 서술꼴이며 전문 용어로 쓰입니다. 뜻은 닿아 있어도 벡터는 멀 수 있습니다. 리랭킹은 찾아온 것 중에서 다시 고르므로, 처음에 후보에 들지 못한 문서는 구할 수 없습니다. 하이드는 처음부터 더 잘 찾으려는 방법입니다.

1) 가상 문서로 검색하기

정의

- 하이드(HyDE, Hypothetical Document Embedding): 가상 문서 임베딩. 질문에 대한 가상의 답변 문서를 LLM으로 만들고, 그것의 임베딩으로 검색하는 방법.
- 가상 문서(hypothetical document): 검색 없이 LLM이 자기가 아는 것만으로 쓴 답. 틀린 내용(할루시네이션)이 섞여 있을 수 있지만 최종 답이 아니므로 괜찮습니다. 검색어로만 씁니다.
- 할루시네이션(hallucination, 환각): LLM이 사실이 아닌 내용을 그럴듯하게 지어내는 것.

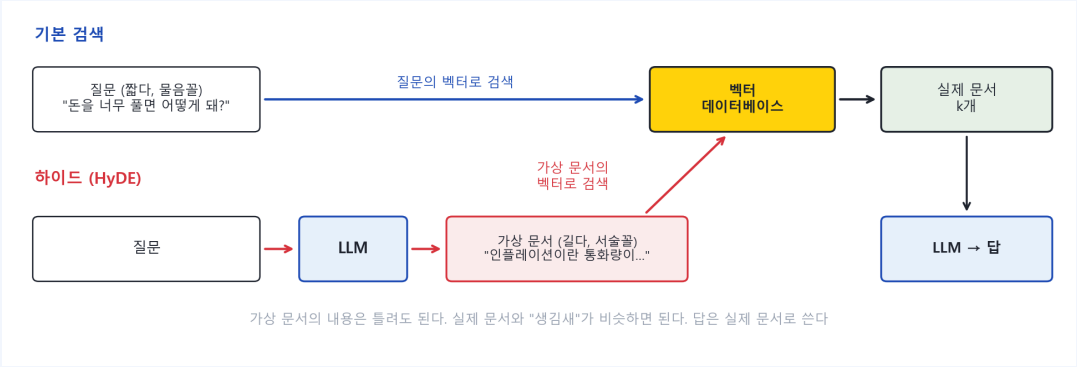


그림 3 기본 검색은 질문의 벡터로, 하이드는 가상 문서의 벡터로 검색한다

단계	하는 일	LLM 호출
① 가상 문서 생성	질문 q를 LLM에 넣어 가상 문서 $\tilde{d}$를 받는다	1회
② 임베딩과 검색	$\tilde{d}$를 임베딩해 가까운 실제 문서 k개를 찾는다	
③ 최종 답 생성	찾은 실제 문서와 원래 질문 q로 답을 쓴다	1회

$$\text{기본 검색: } \underset{d}{\text{top-}k}\ \cos\big(f(q),\ f(d)\big) \qquad \text{하이드: } \underset{d}{\text{top-}k}\ \cos\big(f(\tilde{d}),\ f(d)\big),\quad \tilde{d} = \text{LLM}(q)$$

코사인 거리. 크로마는 유사도 대신 거리를 돌려줍니다. 코사인 거리 = 1 − 코사인 유사도이므로 0에 가까울수록 비슷합니다(24-03). 실습의 "거리"가 이것입니다.

장점	주의할 점
질문의 뜻이 넓어진다. 가상 문서에 관련 용어가 함께 들어간다	가상 문서의 품질에 달려 있다. LLM이 주제를 잘못 짚으면 검색도 엉뚱한 곳으로 간다
표현이 달라도 뜻이 가까운 문서를 더 잘 찾는다	LLM 호출이 1회 늘어난다
	분야(법률, 의료, 금융)에 맞게 가상 문서의 문체를 프롬프트로 맞춰 줘야 한다

예시 적용

일상어로 된 질문 둘(출력에서는 A가 먼저 나옵니다)을 질문 그대로 검색한 결과와, 가상 문서로 검색한 결과를 나란히 놓았습니다(실습 3).

질문 B: "나라끼리 돈을 서로 맡겨 두고 급할 때 꺼내 쓰는 약속이 뭐야?" (용어집에는 "통화스왑"과 "지역금융협정", "글로벌금융안전망" 항목이 있습니다)

	찾아온 청크 (앞부분)
질문으로 바로 검색	지급(payment) 항목 / 연방준비제도 / 외환보유액 / 예약자금이체제도
가상 문서로 검색	금융안전망(지역금융협정, IMF 자금지원) / **글로벌금융안전망** / **지역금융협정** / 대외지급준비자산

질문 그대로는 "돈", "맡겨"에 끌려 지급, 이체 제도를 찾았습니다. 가상 문서("국가 간의 긴급 상황을 대비하여 돈을 서로 맡겨두고 … 국제 금융 관계에서 … 국가 간의 협력과 지원을 위한 중요한 제도")로 검색하자 "국가 간", "금융", "협력" 같은 말이 들어가 금융안전망과 지역금융협정 항목을 찾아왔습니다. 첫째 청크에는 "통화스왑"이라는 말도 들어 있습니다. 최종 답은 "글로벌금융안전망"이었습니다. 하이드가 제 역할을 한 경우입니다.

질문 A: "돈을 너무 많이 풀어서 화폐 가치가 떨어지면 어떻게 돼?" (기대한 것은 인플레이션 항목)

	찾아온 청크 (앞부분)
질문으로 바로 검색	통화승수·지급준비율 / 정책금리와 유동성(유동성 함정) / 유동성의 뜻 / 화폐발행
가상 문서로 검색	정책금리와 유동성(유동성 함정) / 실질구매력 감소 / 통화승수·지급준비율 / 디플레이션

이번에는 둘 다 인플레이션 항목을 찾지 못했습니다. 가상 문서가 "화폐 가치가 떨어지는 것은 금융 상황이 악화되는 것"이라는 엉뚱한 방향으로 쓰였고, 그 결과 최종 답도 "유동성 함정이 발생하면 … 물가는 하락합니다"로 질문과 반대 방향의 이야기가 됐습니다. LLM이 주제를 잘못 짚으면 하이드도 검색을 잘못 이끈다는 "주의할 점"이 그대로 나타났습니다.

거리의 숫자는 비교하지 마세요. 가상 문서로 검색했을 때의 거리(0.43~0.58)가 질문으로 검색했을 때(0.69~0.83)보다 작지만, 이것은 "더 잘 찾았다"는 뜻이 아닙니다. 가상 문서는 질문보다 길고 문서처럼 생겨서 어떤 청크와도 거리가 가까워지는 경향이 있습니다. 질문 A에서 거리는 줄었지만 결과는 나아지지 않았습니다. 검색이 나아졌는지는 찾아온 청크를 읽어 보고 판단해야 합니다.

비유

도서관에서 책 찾기

"돈이 너무 많이 풀리면 어떻게 되지?"라고 검색창에 그대로 치면 잘 안 나옵니다. 사서는 먼저 "아, 통화량과 인플레이션 이야기구나" 하고 그 주제의 책에 나올 법한 말로 바꿔서 찾습니다. 사서의 짐작이 조금 틀려도, 그 서가에 가면 맞는 책이 있습니다.

내가 한 말	= = 질문
사서가 바꿔 말한 것	= = 가상 문서
서가에서 꺼낸 실제 책	= = 검색된 실제 문서
사서가 주제를 잘못 짚는 경우	= = 하이드가 검색을 망치는 경우

실습 3

질문으로 검색한 결과와 가상 문서로 검색한 결과 비교하기

```
# 하이드(HyDE): 질문 대신, LLM이 지어낸 '가상 문서'로 검색한다
from rag_base import ask, generate_answer, load_vector_store

vector_store = load_vector_store()

def generate_hypothetical_doc(question):
    """1단계: 검색 없이, 질문의 답처럼 보이는 가상의 문서를 쓰게 한다."""
    prompt = f"""주어진 질문에 대해, 마치 경제금융 용어 사전에서 발췌한 것 같은
이상적인 답변을 두세 문장으로 작성해주세요.

질문: {question}
```

```
답변:"""
    return ask(prompt, max_new_tokens=120)

def search(text, k=4):
    """2단계: 주어진 글과 가까운 실제 문서를 찾는다. score는 코사인 거리."""
    results = vector_store.similarity_search_with_score(text, k=k)
    return [{"doc": doc, "score": score} for doc, score in results]

def show(results):
    for i, r in enumerate(results, 1):
        text = r["doc"].page_content.replace("\n", " ")
        print(f"  문서 {i} (거리 {r['score']:.4f}) {text[:62]}")

def process_query(question):
    """전체 하이드 과정: 가상 문서 → 검색 → 최종 답변"""
    hypothetical_doc = generate_hypothetical_doc(question)
    relevant_docs = search(hypothetical_doc)
    final_answer = generate_answer(question, [r["doc"] for r in relevant_docs[:2]])
    return {"hypothetical_doc": hypothetical_doc, "retrieved_docs": relevant_docs,
            "final_answer": final_answer}

for question in ["돈을 너무 많이 풀어서 화폐 가치가 떨어지면 어떻게 돼?",
                 "나라끼리 돈을 서로 맡겨 두고 급할 때 꺼내 쓰는 약속이 뭐야?"]:
    print(f"\n질문: {question}")
    print("[질문으로 바로 검색]")
    show(search(question))
    result = process_query(question)
    print("[가상 문서]", result["hypothetical_doc"])
    print("[가상 문서로 검색]")
    show(result["retrieved_docs"])
    print("[최종 답변]", result["final_answer"])
```

실행 결과

```
질문: 돈을 너무 많이 풀어서 화폐 가치가 떨어지면 어떻게 돼?
[질문으로 바로 검색]
  문서 1 (거리 0.6880) 통화승수가 줄어들기 때문에 통화량이 감소한다. 그래서 중앙은행은 통화완화가 필요한  경우에는
지급준비율 인하를
  문서 2 (거리 0.6938) 중앙은행은 정책금리를 낮추고 유동성을 공급한다. 그러나 금리를 계속 낮추는데도  경기가 회복되지
않고 이 이후에
  문서 3 (거리 0.7066) 213 ㅇ  “한국은행이 실시한 유동성 공급 정책” 등과 같은 예에서 유동성은 통화 그 자체를
의미한다.  연관
  문서 4 (거리 0.7078) 화폐발행은 중앙은행인 한국은행의 독점적인 권한으로 금융시장에서 국공채 매입 ,  은행에 대출,
외환 매입, 금융
[가상 문서] 일반적으로, 화폐 가치가 떨어지는 것은 금융 상황이 악화되는 것을 의미합니다. 이는 주로 통화의 공급량이
과도하게 증가하거나, 경제적 불안정성과 같은 요인들로 인해 발생할 수 있습니다. 이러한 상황에서는 화폐 가치가
하락함으로써, 돈을 충분히 사용하기 위해 필요한 자금이 부족해질 수 있습니다. 따라서, 돈을 너무 많이 풀면서 화폐 가치
[가상 문서로 검색]
  문서 1 (거리 0.4258) 중앙은행은 정책금리를 낮추고 유동성을 공급한다. 그러나 금리를 계속 낮추는데도  경기가 회복되지
않고 이 이후에
  문서 2 (거리 0.4330) 국민경제의 실질구매력이 떨어져 실질소득이 감소하고 경상수지가 악화될 수 있다 .  연관검색어 :
무역지수, 수출
  문서 3 (거리 0.4440) 통화승수가 줄어들기 때문에 통화량이 감소한다. 그래서 중앙은행은 통화완화가 필요한  경우에는
지급준비율 인하를
  문서 4 (거리 0.4874) 경기가 침체하고 물가가 하락한다. 만약 경기가 계속 침체하여 디플레이션이 발생하면  명목이자율이
일정할 경우 실
[최종 답변] 유동성 함정이 발생하면 일반적으로 총수요가 감소하여 경기가 침체되고 물가는 하락합니다. 또한, 실질이자율이
상승하여 실질소득 감소와 경상수지 악화가 발생할 수 있습니다. 이러한 상황에서 돈을 너무 많이 풀면 화폐 가치가
떨어집니다.

질문: 나라끼리 돈을 서로 맡겨 두고 급할 때 꺼내 쓰는 약속이 뭐야?
[질문으로 바로 검색]
  문서 1 (거리 0.7972) 결제하고 있다.  연관검색어 : 분리결제, 외환동시결제 (PVP) 지급 지급(payment)은 개인이나
기업
  문서 2 (거리 0.8035) 대한 업무감독, 국고대리업무 등 연방준비제도의 일상적인 업무를 수행한다 .  연관검색어 :
중앙은행 연불수출 무
  문서 3 (거리 0.8162) 위해 언제든지 사용할 수 있도록 보유하고 있는 대외지급준비자산을 외환보유액이라고  한다.
외환보유액은 긴급 시
  문서 4 (거리 0.8324) 결제하도록 하는 제도를 예약자금이체제도라 한다. 한은금융망에서는 두 가지 종류의
예약자금이체제도를 운영하고 있
[가상 문서] "국가 간의 긴급 상황을 대비하여 돈을 서로 맡겨두고 필요할 때 사용하는 약속"이라고 설명하면 됩니다. 이는
국제 금융 관계에서 자주 발생하는 현상 중 하나로, 국가 간의 협력과 지원을 위한 중요한 제도입니다.
[가상 문서로 검색]
  문서 1 (거리 0.5437) 원의 금융안전망, IMF와 같은 범세계적 통화금융기구의 자금지원제도를 포괄하는 글로 벌차원의
금융안전망 등으로
  문서 2 (거리 0.5622) 글로벌금융안전망 금융위기 등으로 인하여 외화유동성이 부족해지는 경우에 대비하기 위하여 각국이
외화유동성을 인출
  문서 3 (거리 0.5652) 지역금융협정 인접국가나 특정 경제권역내 국가간에 체결하는 금융협력협정을 말한다. 1970년대부
```

```
터 시작된 지역금
  문서 4 (거리 0.5842) 준비자산 즉 금, SDR, IMF position 및 보유외화를 말한다. 이러한 대외지급준비자산은  외국의
경제
[최종 답변] 글로벌금융안전망
```

직접 해 보기 일상어로 된 질문을 직접 만들어 넣어 보세요. 가상 문서에 어떤 용어가 들어갔나요? 그 용어 덕분에 검색 결과가 좋아졌나요, 아니면 엉뚱한 곳으로 갔나요?

확인 문제

25-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 하이드에서 LLM이 쓴 가상 문서가 그대로 사용자에게 가는 최종 답이다. (O / X) 이름 _____

2 가상 문서에 틀린 내용이 조금 섞여 있어도 검색어로는 쓸 수 있다. (O / X) 이름 _____

3 하이드에서 벡터 데이터베이스를 검색할 때 쓰는 벡터는? 이름 _____

① 질문의 임베딩 ② 가상 문서의 임베딩 ③ 최종 답의 임베딩

4 코사인 유사도가 0.8일 때 코사인 거리는? 이름 _____

① 0.2 ② 0.8 ③ 1.8

5 LLM이 사실이 아닌 내용을 그럴듯하게 지어내는 것을 _____이라고 한다. 이름 _____

6 기본 RAG와 비교해 하이드는 LLM 호출이 _____회 더 필요하다. 이름 _____

7 리랭킹과 하이드는 검색의 서로 다른 곳을 고칩니다. 각각 어디를 고치는지, 함께 쓸 수 있는지 설명해 보세요. 이름 _____

원문: 25-03 검색 성능을 높이는 하이드(Hyde) — https://wikidocs.net/287282 (교재는 gpt-4o로 가상 문서와 답을 만들고 하이드의 결과만 보여 줍니다. 여기서는 로컬 LLM을 썼고, 차이가 보이도록 질문으로 바로 검색한 결과를 나란히 실었습니다. 데이터와 질문은 금융 용어집으로 바꿨습니다)

오늘의 요약

다시 보기



한눈에

	LLM 리랭킹	크로스 인코더 리랭킹	하이드
고치는 곳	검색의 뒤 (후보의 순서)	검색의 뒤 (후보의 순서)	검색의 앞 (무엇으로 검색할지)
쓰는 모델	LLM + 채점 프롬프트	채점용으로 학습된 작은 모델	LLM + 임베딩 모델
추가 비용	후보마다 LLM 호출 1회	후보마다 작은 모델 1회	LLM 호출 1회
못 고치는 것	1차 검색의 후보에 들지 못한 문서는 구할 수 없다		LLM이 주제를 잘못 짚으면 검색도 틀린다

정의 모음

개념	정의	예시
리랭킹	1차 검색의 후보를 다시 채점해 순서를 바꾸는 것	4개 → 2개, 20개 → 3개
이중 인코더	질문과 문서를 따로 벡터로 바꿔 비교	24장의 벡터 검색
크로스 인코더	질문과 문서를 함께 넣어 점수 하나를 낸다	고양이의 수명: 0.999 / 0.095
하이드 (HyDE)	LLM이 쓴 가상 문서의 임베딩으로 검색	질문 → 가상 문서 → 검색
코사인 거리	1 − 코사인 유사도. 0에 가까울수록 비슷하다	크로마의 score

다음

다음 자료: 26장 LLM 파인 튜닝 (추가 자료)

25장은 검색을 고쳤습니다. 그래도 24-07에서 본 것처럼 작은 LLM은 "답이 없으면 없다고 하라"는 지시를 어깁니다. 다음 자료에서는 LLM 쪽을 고칩니다. 가중치의 1%도 안 되는 작은 행렬(로라)만 학습시켜 모델에게 형식과 규칙을 가르칩니다.

정답과 해설

25-01 LLM 기반의 리랭킹

1. X — 1차 검색이 추린 후보 몇 개만 다시 채점합니다.
2. O — 후보마다 1회씩 늘어납니다.
3. ③ — 채점 4회 + 답 생성 1회
4. ②
5. 리랭킹
6. 2
7. 예시 답: 작은 LLM은 "백틱은 쓰지 마십시오"라는 지시를 어기고 넷 중 셋에 ```` ```json ```` 을 붙였다. 정규식으로 답에서 숫자만 찾도록 해 두었기 때문에 점수를 읽을 수 있었다. LLM의 출력은 지시한 형식을 늘 지키지는 않으므로, 프로그램이 그 출력을 받아 쓸 때는 형식이 어긋나도 동작하도록(또는 실패를 알아채도록) 대비해야 한다.

25-02 크로스 인코더 기반의 리랭킹

1. X — 질문이 있어야 계산할 수 있습니다.
2. O
3. ③
4. ③
5. 이중
6. 20
7. 예시 답: 이중 인코더에서는 질문과 문서가 각자 벡터 하나로 압축된 뒤에야 만나므로, 문서의 어느 부분이 질문에 답하는지 같은 세부가 사라진다. 크로스 인코더에서는 한 입력 안에서 질문의 토큰과 문서의 토큰이 셀프 어텐션으로 서로를 직접 본다.

25-03 검색 성능을 높이는 하이드(HyDE)

1. X — 검색어로만 씁니다. 최종 답은 검색된 실제 문서로 다시 씁니다.
2. O
3. ②
4. ① — 1 − 0.8
5. 할루시네이션 (환각)
6. 1
7. 예시 답: 하이드는 검색하기 전에 "무엇으로 검색할지"를 고치고, 리랭킹은 검색한 뒤에 "찾아온 것의 순서"를 고친다. 서로 다른 단계라 함께 쓸 수 있다. 가상 문서로 넓게 찾은 뒤 크로스 인코더로 다시 고르면 된다.

용어 정리

용어	정의
검색기 (리트리버)	질문과 관련된 문서를 찾아 주는 부품
리랭킹	1차 검색이 돌려준 후보를 다른 모델로 다시 평가해 순서를 다시 매기는 것
2단계 검색	빠른 방법으로 넓게 찾고, 정확한 방법으로 몇 개만 다시 고르는 구조
이중 인코더	질문과 문서를 각각 벡터로 바꾼 뒤 유사도를 구하는 방식
크로스 인코더	질문과 문서를 한 입력으로 넣어 관련성 점수를 바로 내는 방식
하이드 (HyDE)	질문에 대한 가상의 답변 문서를 만들어 그것의 임베딩으로 검색하는 방법
가상 문서	검색 없이 LLM이 쓴 답. 검색어로만 쓴다
할루시네이션	LLM이 사실이 아닌 내용을 그럴듯하게 지어내는 것
코사인 거리	1에서 코사인 유사도를 뺀 값